

Topic 1

Authentication Protocols from the Wire Up

This is the session teaching document. Every concept is built from first principles - no assumed knowledge, every term defined before it is used, every mechanism shown rather than gestured at. Each protocol section opens with an analogy, then states exactly where the analogy stops being true, then shows the mechanism as a diagram and as prose.

After reading this document you will be able to explain - without notes - what is in the packet when a user authenticates, who signs what, who trusts whom, and why the modern cloud identity model defaults to bearer tokens and what that costs you.

TOPIC	1 of 7 - Phase 1
CERTIFICATION	SC-300 - Identity and Access Administrator Associate
SESSION TYPE	Teaching session - study material
PREREQUISITES	None. This is the foundation.
XP AVAILABLE	200 XP - Quiz 60 · Build Lab 50 · Forensic Lab 50 · Artifact 40

How to use this document

- **Section 01 comes before any protocol content on purpose.** Seal your five lab predictions there before reading further. Once you have read Sections 05 and 06, three of the five are no longer predictions.
- **Each protocol ends with a STOP box.** Answer its questions before turning the page. Answering is the study; reading is not.

- **Each section ends with a VERIFY box** naming the primary source and section to check. This document was drafted by an AI and its first version contained ten factual errors. Treat every load-bearing claim as unverified until you have read the cited source yourself. That reading is not overhead - it is the study.

The Vocabulary - Defined Once, Used Everywhere

Most confusion in identity and authentication is vocabulary confusion wearing a technical costume. Before we touch a single protocol, we define every term that will do load-bearing work in this topic. Learn these definitions precisely. Imprecise vocabulary produces imprecise thinking.

Principal

Any entity an identity system can name and authenticate. A human user. A computer account. A service account. An application. The word 'principal' does not mean 'user' - a service running without a human present is still a principal. Get this wrong and you will misclassify half your identity estate.

Credential

Something a principal presents to prove it is that principal. Credentials fall into three categories: something you know (a password), something you have (a hardware key, a certificate, a smartcard), or something you are (a biometric). Strong authentication combines two of the three.

Claim

A statement about a principal, asserted by an issuer. The email address `avidal@company.com` is a claim. The group membership 'Domain Admins' is a claim. The role 'Global Administrator' is a claim. A claim is only as trustworthy as the issuer that signed it - which is why we care intensely about who issued a token.

Token / Assertion

A signed container of claims. 'Assertion' is SAML's word; 'token' is OAuth/OIDC's word. Same concept, different serialization (XML versus JSON). A token says: 'I, the issuer, assert that this principal has these claims, and I have signed this so you can verify I said it.'

Authentication (AuthN)

Establishing which principal this is. 'AuthN' is shorthand for authentication, used so it cannot be confused with the next term. It answers: who are you?

Authorization (AuthZ)

Establishing what that principal may do. 'AuthZ' is shorthand for authorization. It happens after authentication and depends on it. It answers: now that we know who you are, what are you allowed to do?

Signing

Applying a cryptographic operation such that a verifier can confirm (a) the content was not altered and (b) it came from the holder of a specific key. Critical: signing does not hide anything. A signed JWT is readable by anyone who intercepts it. Signing proves authenticity and integrity, not confidentiality.

Bearer token

A token where possession alone is sufficient. Whoever holds it, is it. No additional proof is required that the holder is the principal the token names. If I steal your bearer token, I am you, for all practical purposes, until it expires.

Proof-of-possession (PoP) token

A token that additionally requires demonstrating control of a cryptographic key. Stealing the token alone is not enough - the attacker must also control the key. Significantly harder to abuse.

KEY POINT - Bearer vs Proof-of-Possession

The distinction between bearer tokens and proof-of-possession tokens is the single most consequential idea in this topic. Everything in Topic 3 - Conditional Access, token lifetimes, Continuous Access Evaluation, session replay - exists because the modern cloud protocols default to bearer tokens. Write this down: bearer token = possession is sufficient.

SECTION 01

Seal Your Predictions - Before Reading Any Further

CRITICAL - Do This Now, Not Later

The build lab is worth 50 XP and requires five predictions recorded before you touch the tenant. Sections 05 and 06 of this document answer three of the five directly. If you read them first, you are recalling, not predicting, and the lab loses its value. Write your five answers now, with your reasoning, in a file. Save it. Do not edit it afterward. If you have already read ahead, record that honestly in the file - a compromised prediction you disclose is worth more than one you pretend was sealed.

You will register an application in your dev tenant and run an authorization code + PKCE flow against it. Predict, with reasoning, using only what you know today:

Prediction 1. You request the scopes `openid profile User.Read`. How many tokens come back, and what is the `aud` claim of each?

Prediction 2. Will the access token be a decodable JWT or an opaque string? What does that imply about what your client is allowed to do with it versus what it can do?

Prediction 3. The nonce you send in the request - which token does it come back in, and why that one and not the other?

Prediction 4. You now request only `openid profile`, dropping `User.Read`. Do you still get an access token? If so, who is it for?

Prediction 5. Predict the difference between the `scp` claim and the `roles` claim. Which would you expect in a token obtained by a signed-in user, versus by an application with no user present?

KEY POINT - The Prediction Is the Work

A wrong prediction you can explain is a full pass. No prediction is zero, however well the build goes. The prediction forces you to consult your mental model before reality corrects it. The delta is where learning happens.

Predictions sealed? Turn the page.

Kerberos - The On-Premises Answer

Kerberos is the authentication protocol that underpins on-premises Windows environments. Every time a user logs into a domain-joined machine and accesses a file server, a database, or a print queue - Kerberos is what makes that work. It was designed at MIT in the late 1980s as part of Project Athena, standardized as version 5 in 1993 (RFC 1510, later RFC 4120), and became Windows' native protocol with Windows 2000 and Active Directory.

The Problem Kerberos Was Built to Solve

Imagine a university campus in the 1980s. Students need access to many different computers and services across the network. How do you let a student authenticate to the library catalog, the printing service, and the email server - without sending their password to each of those services, and without each service having to contact a central server every time someone makes a request?

Three properties were required:

- The user's password must never cross the network in any form
- The service being accessed must never learn the user's password
- The service must be able to verify the user without calling a central server at the moment of access

That third property is the design achievement that makes Kerberos scale. If every service had to contact a central authority every time someone made a request, that authority becomes a single point of failure and a bottleneck for the entire network.

ANALOGY - The Concert Wristband System

Think of a large music festival with multiple stages and VIP areas. When you arrive, you show your ticket to the main gate (the KDC). They verify you and give you a wristband (the TGT). When you want the VIP area (a specific service), you show your wristband to the VIP booth and they give you a special VIP wristband (the service ticket). You show that to the VIP guard (the service). The guard can verify it without calling the main gate. Your original ticket (password) never leaves your pocket after check-in. The guard never sees it. The VIP booth issued a credential scoped to exactly that one area.

Where this analogy breaks: A wristband can be read by anyone and works for whoever wears it - that would make it a bearer token, and a Kerberos ticket is not one. The real guard also makes you prove you hold a secret that was issued alongside the wristband (the session key, proven by the authenticator). And real tickets are encrypted: the client cannot read its own TGT. Keep the flow from the analogy; do not keep its trust model.

The Three Parties and What They Hold

Party	Who They Are	What They Hold
Client	The user or machine authenticating	A long-term key derived from the user's password. After initial authentication, the TGT and a session key.
KDC - Key Distribution Centre	On Windows: every writable domain controller. It has two logical halves: the AS (Authentication Service) and the TGS (Ticket Granting Service).	The long-term key for every principal in the domain - users, computers, service accounts - plus the krbtgt account key. It is the master of the whole system.
Service	The resource being accessed - a file server, database, web app, anything registered with an SPN	Its own long-term key only. Nothing about the user. Nothing about other services. This is what makes it scale.

The Flow as a Diagram

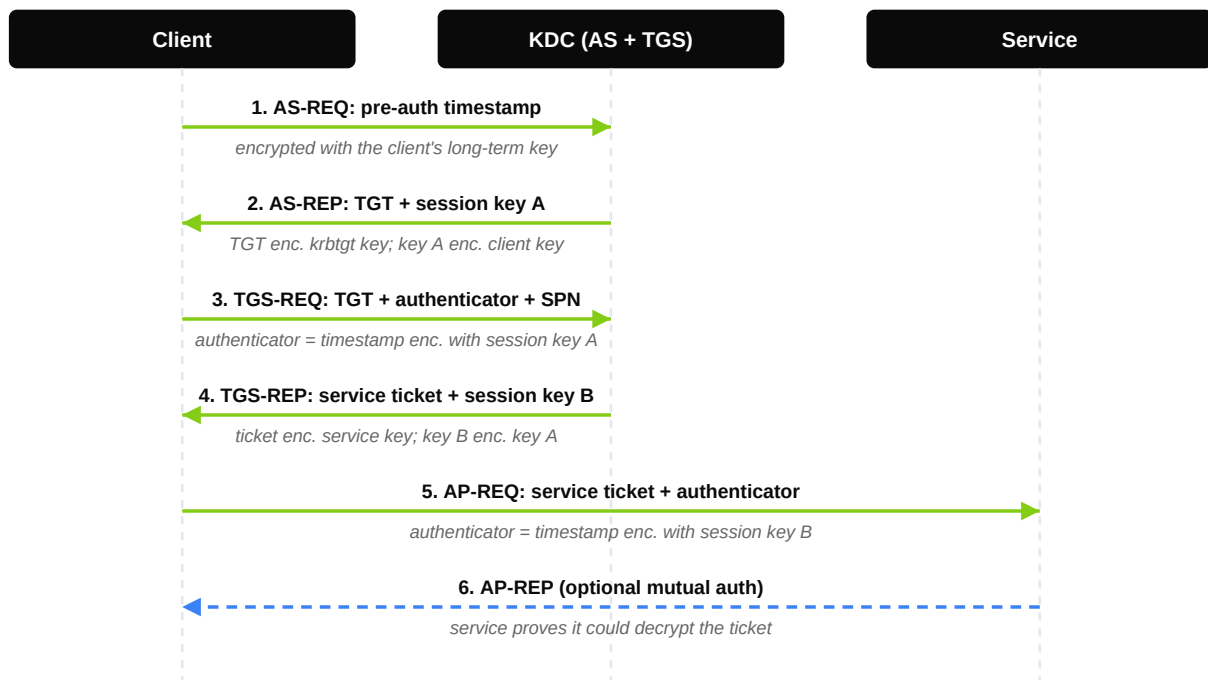


Figure 1. Kerberos: the KDC is out of the picture after step 4. The service never contacts it.

The Flow as Prose

Step 1 - AS-REQ / AS-REP: Getting the Ticket Granting Ticket (TGT)

When a user logs in, their machine sends an AS-REQ (Authentication Service Request) to the KDC. To prove they know their password without sending it, the client encrypts a current timestamp using a key derived from the password. This is pre-authentication. The KDC already holds that same long-term key for the account. It decrypts the timestamp and verifies it is current - within five minutes by default, which is why clock skew breaks Kerberos.

If pre-authentication succeeds, the KDC sends back two things:

- **The TGT** - a Ticket Granting Ticket, encrypted with the krbtgt account's long-term key. The client cannot read it. Only the KDC can. Think of it as a sealed envelope the KDC handed you.
- **Session key A** - encrypted with the client's own long-term key, so the client can decrypt it. It is used to talk to the KDC in step 2.

Step 2 - TGS-REQ / TGS-REP: Getting a Service Ticket

Now the user wants a specific service - say, a file server. They name it by its SPN (Service Principal Name - a unique identifier like MSSQLSvc/db01.company.com:1433). The client sends a TGS-REQ to the KDC, presenting the sealed TGT along with an authenticator: a timestamp encrypted with session key A, proving the client actually holds that key.

The KDC opens the TGT (only it can), verifies the authenticator, and creates a service ticket encrypted with the target service account's long-term key, plus a new session key B for the client to use with the service.

Step 3 - AP-REQ: Presenting the Service Ticket to the Service

The client sends the service an AP-REQ containing the service ticket and a fresh authenticator - a timestamp encrypted with session key B. The service decrypts the ticket with its own long-term key, finds the client's identity and session key B inside, then uses key B to decrypt the authenticator. If the timestamp is valid, the service knows two things: this ticket was issued by the KDC for me, and the party presenting it holds the session key issued alongside it.

KEY POINT - The Authenticator Is Why a Kerberos Ticket Is Not a Bearer Token

Possessing the service ticket alone is not enough - the presenter must also prove it holds the session key by producing a valid authenticator. This is proof-of-possession. The caveat: if an attacker steals both the ticket and the session key from memory (pass-the-ticket), they can produce a valid authenticator too. Kerberos is not immune to theft; it simply requires stealing more than a single string.

THE CORE ACHIEVEMENT

The service verified the user without ever contacting the KDC. The KDC's involvement ended at step 4 of the diagram. This is why Kerberos scales across thousands of services and millions of authentications.

SPN - Service Principal Name - Why It Matters

The SPN is how the KDC knows which service account's key to use when encrypting a service ticket. Format examples:

- MSSQLSvc/db01.company.com:1433 - SQL Server on db01
- HTTP/webserver.company.com - a web application
- HOST/computer01.company.com - the machine account

If an SPN is missing, Kerberos cannot proceed and Windows falls back to NTLM (next section). If an SPN is duplicated - registered on two accounts - Kerberos fails outright. SPN problems are one of the most common causes of silent Kerberos failures in real environments.

Kerberoasting - The Attack That Follows Directly from the Design

Any authenticated domain user can request a service ticket for any SPN. That ticket is encrypted with the service account's long-term key - which is derived from its password.

CRITICAL - Kerberoasting

An attacker who has compromised any domain account can request service tickets for every service account in the domain, take those encrypted blobs offline, and brute-force the service account passwords at leisure. No alert fires. No failed logon is generated. No lockout occurs. The only signal is the service ticket request in event ID 4769 on the domain controller - and most environments do not alert on that.

The cost of this attack depends entirely on the encryption type (etype) of the ticket:

Etype	Algorithm	Offline Cracking Cost	Status
0x17	RC4-HMAC	LOW - the key is the unsalted NT hash of the password; candidates are fast to test	Removed as a default in July 2026
0x11	AES128-CTS-HMAC-SHA1-96	HIGH - salted, iterated key derivation	Acceptable
0x12	AES256-CTS-HMAC-SHA1-96	HIGH - strongest standard option	Preferred

Microsoft completed the removal of RC4 as a default in July 2026 under CVE-2026-20833. Any account in your estate still carrying an explicit RC4 override via the msDS-SupportedEncryptionTypes attribute is an exception someone set to stop a breakage - and probably never documented. Finding and documenting those is Topic 1's artifact.

msDS-SupportedEncryptionTypes decimal values to recognize: 4 = RC4 only · 8 = AES128 · 24 (0x18) = AES128 + AES256 · 28 (0x1C) = RC4 + AES128 + AES256.

STOP - Answer these before reading on. Out loud or on paper. No looking back.

1. Trace step 5 of Figure 1. Name every key involved and who holds each one. Why can the service decrypt the ticket but the client cannot?
2. A colleague says 'if someone steals a Kerberos service ticket from the network they can replay it.' What is missing from that claim, and what would the attacker also need?
3. Your SQL service account has msDS-SupportedEncryptionTypes = 28. Is that a problem? What would an attacker do with it?

VERIFY - Do not trust this document. Check it.

RFC 4120 §3.1 (AS exchange), §3.3 (TGS exchange), §3.2 (AP exchange - read §3.2.2 for what goes in the authenticator). Microsoft Support: 'How to manage Kerberos KDC usage of RC4' (CVE-2026-20833) for the enforcement timeline and registry values.

SECTION 03

NTLM - The Thing Kerberos Was Meant to Replace

NTLM predates Kerberos in Windows. It was the native Windows authentication protocol from Windows NT (1993) until Windows 2000 and Active Directory made Kerberos the default. Kerberos itself is the older design - MIT built it in the 1980s - but Windows did not adopt it until 2000. NTLM still exists everywhere in Windows environments, not because it is good, but because it is the fallback when Kerberos fails, and Kerberos fails more often than people realize.

ANALOGY - The Phone PIN Verification

When you call your bank, the agent asks a question only you should be able to answer. You answer it. They check the answer against their records and decide whether you are who you say you are. That is NTLM: you prove you know the secret by demonstrating knowledge of it. Two problems. First, you never verified you were actually talking to your bank - anyone who dials you and asks the question can collect your answer. Second, if someone records your answer, they can call the real bank and replay it.

Where this analogy breaks: NTLM never sends the secret itself - it sends a value computed from the password hash and the server's challenge, so an eavesdropper does not learn the hash from one exchange. The analogy overstates the leakage. What it gets right is the two structural gaps: no proof of who you are talking to, and nothing binding the answer to that specific call.

The Flow as a Diagram

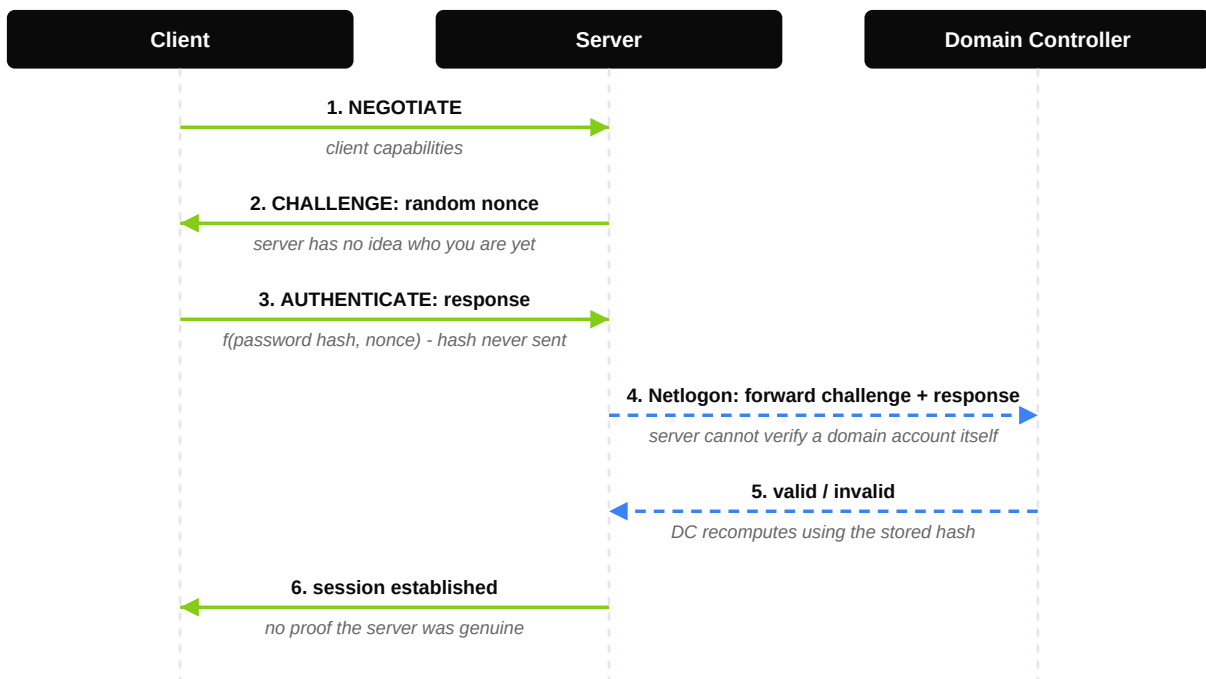


Figure 2. NTLM: three messages, then the server outsources verification to a DC. Nothing ties the response to this server.

The Flow as Prose

- **NEGOTIATE** - client tells the server which NTLM features it supports
- **CHALLENGE** - server sends a random number (a nonce) to the client
- **AUTHENTICATE** - client responds with a value computed from the nonce and a key derived from the user's password hash

The server cannot verify this response itself unless it holds the password hash locally, which is only true for local accounts. For domain accounts, the server forwards the challenge and response to a domain controller through the Netlogon service, and the DC checks the response against the stored hash.

Three Structural Defects - Not Bugs, Design Choices That Aged Badly

Defect 1: No Mutual Authentication

The client proves it knows the password. The server never proves anything. The client has no way to verify it is talking to the legitimate server, so a rogue server can harvest NTLM responses from any client that connects to it.

Defect 2: No Binding to a Target

The NTLM response proves 'I know the hash' - it does not cryptographically bind that proof to 'I meant to talk to this specific server over this specific connection.' This is what makes NTLM relay possible: attacker A tricks your machine into authenticating to A, then forwards your challenge-response exchange to server C in real time, acting as a middleman. C accepts it - the math is valid, and nothing in the response says it was meant for anyone but C. A now has a session on C as you.

The mitigation is Extended Protection for Authentication (EPA) - channel binding and service binding, which tie the response to the TLS channel and the target name. EPA is not enabled by default on many services, which is why relay remains a working attack in 2026.

Defect 3: Pass-the-Hash

The NTLM response is derived from the password hash, never from the password itself. For NTLM's purposes the hash is the password. If an attacker extracts the hash from memory - tools like Mimikatz do exactly this - they can authenticate as that user without ever learning the plaintext password.

CRITICAL - NTLM Is Deprecated

Microsoft deprecated all versions of NTLM in June 2024. NTLMv1 was removed starting with Windows 11 24H2 and Windows Server 2025. NTLMv2 remains present and working, but deprecated. If your estate is still running NTLM, it is running a deprecated protocol with three structural defects that have been exploited in real attacks for three decades.

Why NTLM Is Still Running in Your Estate

Windows does not call Kerberos or NTLM directly. It calls Negotiate (SPNEGO), a wrapper that tries Kerberos first and silently falls back to NTLM if Kerberos fails. Kerberos fails when:

- An SPN is missing - the service was never registered correctly
- A duplicate SPN exists - registered on two different accounts
- A connection is made by IP address instead of hostname - Kerberos needs a hostname to look up the SPN
- The target is not a domain member

These failures are not loud. There is no error. The user gets in anyway - via NTLM. Your estate could be running NTLM for dozens of services and you would not know without reading the authentication logs.

STOP - Answer these before reading on. Out loud or on paper. No looking back.

1. In Figure 2, where would an attacker insert themselves to perform a relay, and which two arrows would they copy?
2. A user connects to \\10.0.0.5\share instead of \\fileserver\share. Which protocol authenticates them, and why?
3. Why does pass-the-hash work against NTLM but not directly against Kerberos AES tickets?

VERIFY - Do not trust this document. Check it.

MS-NLMP (Microsoft's open specification for NTLM), §3.3.2 'NTLM v2 Authentication' for how the response is computed. Microsoft Learn: 'Deprecated features for Windows client' for the June 2024 deprecation entry. Microsoft Learn: 'Extended Protection for Authentication' for what EPA binds and where it is off by default.

SAML 2.0 - The Federation Answer

Kerberos and NTLM solve authentication within a single organization with a shared directory. SAML solves a different problem: how do you authenticate a user from one organization into an application run by a completely different organization, with no shared directory, no shared password store, and no network path between them?

ANALOGY - The University Library Card

Your university gives you a student ID. When you visit the city library, they do not have you in their system. But the city library has an agreement with your university: they trust the university to vouch for its students. You show your university-issued ID. The library checks that it looks legitimate - official card format, valid date, the university seal - and lets you in. The library never contacts the university in real time. The trust was established when the two organizations made their agreement. Your student ID is the assertion. The university is the IdP. The city library is the SP.

Where this analogy breaks: A student ID is reusable for years at any library that honours it. A SAML assertion is valid for minutes, is addressed to exactly one SP (the audience restriction), and is normally consumed once. The card also has no cryptographic signature - the library is eyeballing a hologram. The SP verifies a digital signature against a certificate it was given in advance. Same trust shape, much narrower credential.

The Flow as a Diagram

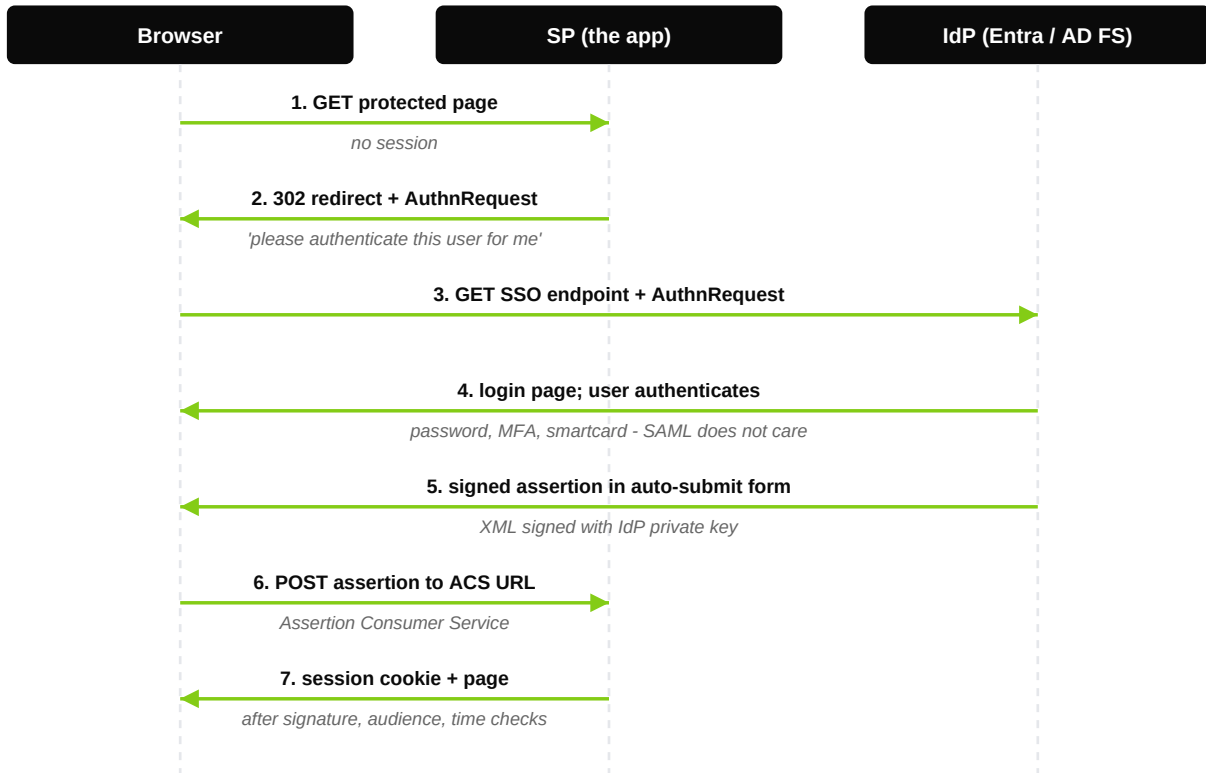


Figure 3. SAML web browser SSO, POST binding. Every arrow touches the browser. SP and IdP never speak directly.

The Three Parties

- **IdP - Identity Provider:** where the user is known and authenticated. In your estate: Microsoft Entra ID, or an on-premises AD FS server.
- **SP - Service Provider:** the application the user wants. Salesforce, ServiceNow, Workday, any SaaS that supports SAML.
- **The browser:** the intermediary. In the POST binding, SAML is a front-channel protocol - everything flows through the browser and the IdP and SP never talk directly. (The less common artifact binding does add a back channel.)

What Is in the Assertion

- Subject - who the user is, in a format the SP understands (NameID)
- Conditions - NotBefore and NotOnOrAfter timestamps; the AudienceRestriction naming the SP this assertion is for
- AuthnStatement - when and how the user authenticated
- AttributeStatement - the claims: email, groups, display name
- Signature - over the assertion, using the IdP's private key

KEY POINT - The Trust Anchor Is a Certificate

The SP trusts the IdP because it holds the IdP's public certificate, exchanged out of band when the federation was configured. This is why SAML federations break when the IdP rotates its signing certificate - the SP is still validating against the old certificate, and every assertion now fails.

Why This Design Matters

Because everything flows through the browser, no direct network path is required between the IdP and the SP. An on-premises AD FS server can federate with a cloud SaaS application with no firewall exceptions - the browser carries the assertion across the public internet. This is what made cross-organization SSO practical.

XML Signature Wrapping - The Attack to Know

SAML assertions are XML. An SP that does not correctly canonicalize and validate the XML signature is vulnerable to signature wrapping: the attacker takes a valid assertion, moves the signed element elsewhere in the XML tree, and inserts malicious claims outside the signed portion. The validator checks the still-intact signed element and reports 'valid.' The application then reads claims from the unsigned portion. This has been exploited in production. The fix is rigorous canonicalization and confirming that the signed element is exactly the element the application reads claims from.

STOP - Answer these before reading on. Out loud or on paper. No looking back.

1. In Figure 3, which arrow carries the thing the SP actually trusts, and what does the SP check it against?
2. The IdP rotates its signing certificate on Friday night. Describe Monday morning from the SP's point of view, arrow by arrow.
3. An assertion for Salesforce is captured and replayed to ServiceNow, which federates with the same IdP. What single field should stop this, and what happens if ServiceNow does not check it?

VERIFY - Do not trust this document. Check it.

OASIS SAML 2.0 Core, §2 (assertions - read 2.5 for Conditions and AudienceRestriction). OASIS SAML 2.0 Profiles, §4.1 'Web Browser SSO Profile' for the exact message sequence in Figure 3.

SECTION 05

OAuth 2.0 - The Distinction Most Administrators Get Wrong

CRITICAL - OAuth 2.0 Is Not an Authentication Protocol

OAuth 2.0 is an authorization framework. It answers: may this application act on this resource on this user's behalf, and to what extent? It does not answer: who is this user? This distinction is tested on SC-300 and is routinely confused by working administrators. Fixing that confusion is one of the primary goals of this topic.

ANALOGY - The Hotel Keycard

You check into a hotel. At reception you prove who you are - you show your passport. The receptionist gives you a keycard. The keycard does not say who you are. It says what you can open: Room 412, the gym, the business centre. It expires on Sunday. When you hold it to the gym door, the door does not call reception to verify your identity. It checks: is this card valid, and does it grant gym access? If you drop it in the lobby and a stranger picks it up, that stranger can use the gym until Sunday. That is a bearer token.

Where this analogy breaks: A keycard is physically bound to one hotel's door system - the equivalent of the aud claim is enforced by the hardware. An access token's audience is just a string, and it only protects you if the API bothers to check it. Also, the front desk can deactivate a lost keycard instantly. Outside of CAE (Section 07), an issuer cannot recall a bearer token it has already issued.

The Four Roles in OAuth 2.0

Role	What It Is	In Your Estate
Resource owner	The user (usually) who owns the data or resource being accessed	The employee signing in and approving the access request
Client	The application that wants to act on the resource on the user's behalf	A custom app, a third-party integration, a service principal
Authorization server (AS)	Issues tokens. Manages consent. The trust anchor.	Microsoft Entra ID
Resource server (RS)	The API that holds the data the client wants to access	Microsoft Graph, your own API, any API protected by Entra ID

Authorization Code Flow with PKCE - The Diagram

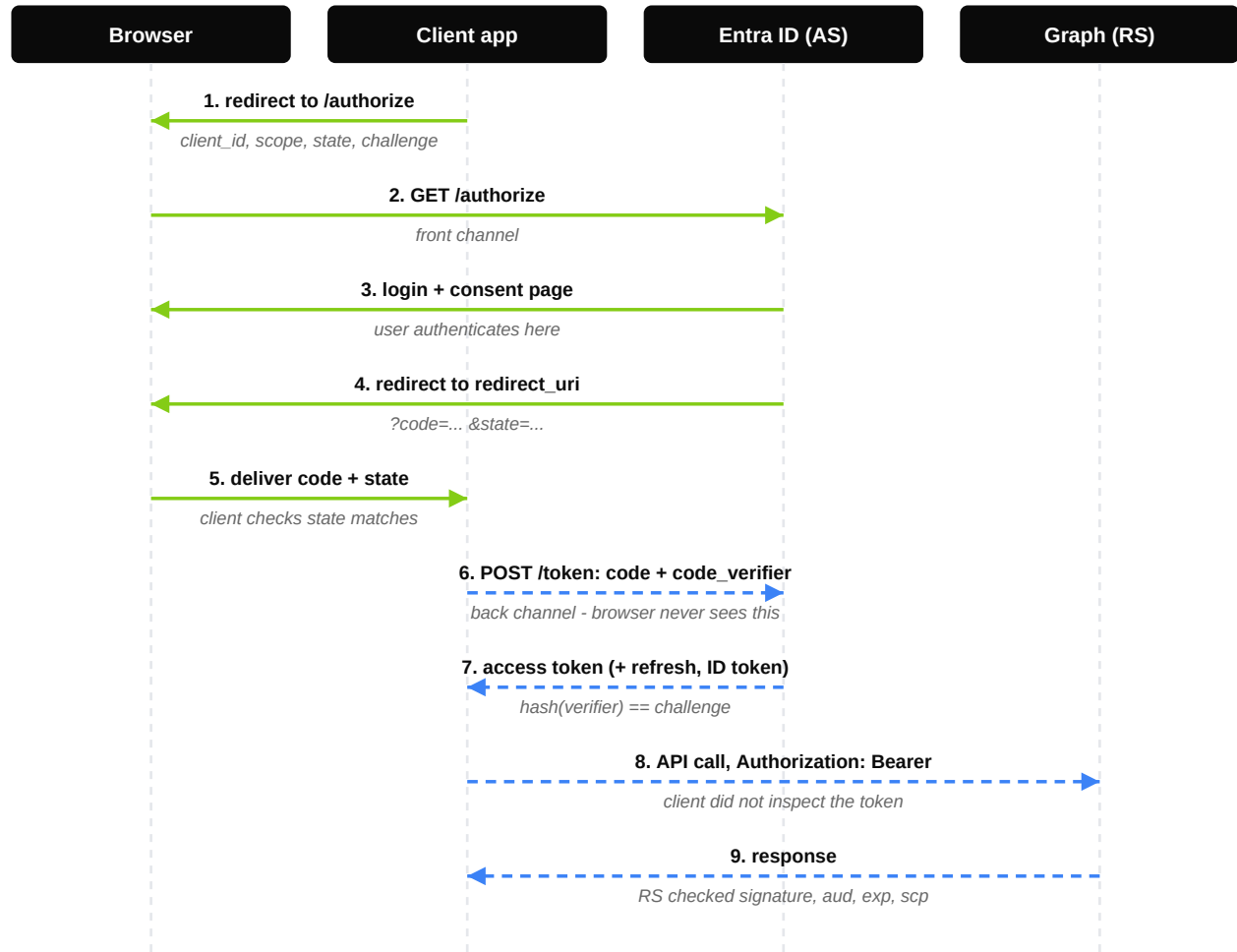


Figure 4. Authorization code + PKCE. Solid = through the browser (front channel). Dashed = server to server (back channel).

The Flow as Prose

- The client generates a random string, the `code_verifier`, and hashes it to produce the `code_challenge`.
- The client redirects the browser to the AS with `client_id`, `redirect_uri`, `scope` (the permissions it wants), `state` (a random value for CSRF protection), and the `code_challenge`.
- The user authenticates at the AS and consents to the requested scopes - or an admin pre-consented for the tenant.
- The AS redirects the browser back to the client's `redirect_uri` with a short-lived authorization code and the state value. The client checks state matches what it sent.
- The client sends the code to the AS on the back channel - server to server, not through the browser - with the original `code_verifier`.
- The AS verifies `hash(code_verifier)` equals the `code_challenge` from step one. This proves the entity redeeming the code is the one that started the flow.
- The AS returns an access token, and optionally a refresh token and an ID token.

- The client sends the access token to the resource server in the Authorization: Bearer header.

PKCE - Proof Key for Code Exchange, pronounced 'pixie' - solves a specific attack. On mobile apps and single-page applications the redirect can be intercepted by a malicious app on the same device. An attacker who intercepts the code cannot redeem it without the code_verifier, which never left the legitimate client.

Scopes - What the Client Is Asking Permission For

A scope is a permission string describing what the client wants to do. Examples from Microsoft Graph:

- User.Read - read the signed-in user's profile
- Mail.Send - send email as the signed-in user
- Directory.ReadWrite.All - read and write all directory data (admin consent required)

Granted scopes appear in the access token: in the scp claim for delegated permissions (a user is present) or the roles claim for application permissions (no user present). The resource server checks these claims before allowing the operation.

The Rule Administrators Break

CRITICAL - Do Not Parse Access Tokens in the Client

Microsoft's documentation is explicit: client applications should treat access tokens as opaque strings. They should not attempt to validate or parse them. The access token is intended for the resource server, not the client. Even though Microsoft's access tokens are currently JWTs and can be decoded at jwt.ms, Microsoft reserves the right to change that format. An application that depends on the contents of an access token is built on sand. (Decoding one to learn from it, as you will in the lab, is fine. Building on it is not.)

RFC 9700 - Current Best Practice (January 2025)

RFC 9700, published as BCP 240, is the current best-practice document for OAuth 2.0 security. It updates RFC 6749, 6750, and 6819. Key positions:

- PKCE for all clients - not optional
- Redirect URI matching must be an exact string match - no wildcards, no prefix matching
- The implicit grant is out - convenient but fundamentally insecure
- The resource owner password credentials (ROPC) grant is out - it hands the user's password to the client
- Sender-constrained tokens are preferred over bearer tokens

That last point connects back to the vocabulary section: the RFC itself recommends moving from bearer tokens toward proof-of-possession. The industry is slowly catching up to the security model the design calls for.

STOP - Answer these before reading on. Out loud or on paper. No looking back.

1. In Figure 4, which arrows would an attacker on the same device be able to observe, and why does PKCE make that observation useless?
2. Why is arrow 6 dashed? What would go wrong if the code were exchanged over arrow 2's channel instead?
3. State and nonce both defend against replay-style attacks. Which one belongs to OAuth, which to OIDC, and what does each one bind?

VERIFY - Do not trust this document. Check it.

RFC 6749 §1.1 (the four roles) and §4.1 (authorization code grant). RFC 7636 (PKCE) §1.1 for the attack it prevents. RFC 9700 §2 for the current recommendations list. Microsoft Learn: 'Access tokens in the Microsoft identity platform' - find the sentence telling clients to treat tokens as opaque.

OpenID Connect - The Authentication Layer OAuth Lacks

We established that OAuth 2.0 is an authorization framework - it does not tell you who the user is. OpenID Connect (OIDC) is a thin layer built on top of OAuth 2.0 that adds exactly that. It is not a separate protocol - it uses the OAuth 2.0 flows and extends them. In Figure 4, adding the scope openid turns the flow into OIDC, and an ID token arrives alongside the access token on arrow 7.

ANALOGY - The Passport Stamp vs the Event Ticket

When you enter a country, the border officer stamps your passport (authentication - this is you, and you have entered). You then go to a concert and show your ticket (authorization - you may enter this venue). The passport (ID token) is about identity: who you are, where you are from, when you arrived. The ticket (access token) is about access rights: which section, which date. The door staff check your ticket, not your passport. The border officer checks your passport, not your ticket. Using the ticket to identify yourself at the border would be wrong - even if it has your name printed on it.

Where this analogy breaks: A passport is good for ten years and accepted by every country. An ID token is good for about an hour and is addressed to exactly one client - and must equal that client's ID or it is rejected. And the border officer may phone the issuing country; the client never phones Entra ID - it verifies the signature locally using published keys.

What OIDC Adds: The ID Token

The ID token is a JWT issued to the client that describes who authenticated. It is for the client. Not for the API.

OpenID Connect Core requires five claims in every ID token, plus a sixth whenever the client sent one:

Claim	Name	What It Contains
iss	Issuer	Who issued this token. Must exactly match the expected authority URL.
sub	Subject	The stable, unique identifier for this user at this issuer for this client. It never changes. Key on this, not on email.
aud	Audience	Must exactly equal the client's own client_id. If it does not, reject the token.
exp	Expiry	Unix timestamp after which the token must be rejected.
iat	Issued at	When the token was issued. Used to reject tokens that are too old.
nonce	Nonce	Required only when the client sent one in the request. Must come back unchanged. This is the replay defence.

THE SINGLE MOST IMPORTANT SENTENCE IN THIS TOPIC

The ID token is for the client and says who the user is. The access token is for the API and says what may be done. They are not interchangeable. A system that treats them as interchangeable is broken even when it appears to work.

Two Real Tokens, Side by Side

Everything above is abstract until you see the actual objects. Below are the decoded payloads of an ID token and an access token issued by Entra ID for the same sign-in - the same human, the same app registration, the same second. Tenant and object IDs are shortened. Read the annotations in green first.

The ID token (v2.0 format) - issued to the client

```
{
  "aud": "6e74172b-be56-4843-9ff4-e66a39bb12e3",

  "iss": "https://login.microsoftonline.com/72f9...db47/v
2.0",
  "iat": 1757246400,
  "nbf": 1757246400,
  "exp": 1757250300,
  "name": "Alexander Vidal",
  "preferred_username": "avidal@contoso.com",
  "oid": "a3f9c2e1-...-8b1d",

  "sub": "AAAAAAAAAAAAAAAAAAIkzqFVrSaSaFHy782bbtaQ",

  "tid": "72f988bf-86f1-41af-91ab-2d7cd011db47",
  "nonce": "8c1e4f7a-2b9d-4e6f",

  "ver": "2.0"
}
```

Your app registration's client_id. The client checks this equals its own ID and rejects the token otherwise.

The v2.0 issuer. Note the /v2.0 suffix - compare with the access token below.

Issued at (Unix time).

Not before. Same as iat here.

Expires 65 minutes later. This token is dead after that.

Display name - a claim, not an identifier.

Human-readable. Can change. Do not key on it.

Directory object ID. Same for this user in every app. This is the identifier to correlate across tokens.

Pairwise subject: unique to this user AND this client. A different app gets a different sub for the same person.

Tenant ID.

The exact value the client sent in the /authorize request. Client checks it matches. Present in the ID token only.

Token format version.

The access token for Microsoft Graph (v1.0 format) - issued for the API

```
{
  "aud": "https://graph.microsoft.com",
  "iss": "https://sts.windows.net/72f9...db47/",
  "iat": 1757246400,
  "nbf": 1757246400,
  "exp": 1757250300,
  "acr": "1",
  "appid": "6e74172b-be56-4843-9ff4-e66a39bb12e3",
  "appidacr": "0",
  "scp": "User.Read profile openid email",
  "oid": "a3f9c2e1-...-8b1d",
  "sub": "kX9pQ2mL...-different-value",
  "tid": "72f988bf-86f1-41af-91ab-2d7cd011db47",
  "upn": "avida1@contoso.com",
  "ver": "1.0"
}
```

Not your app. Graph checks this; your client does not. (Graph tokens sometimes show the GUID 00000003-0000-0000-c000-000000000000 instead - same meaning.)

A different issuer string from the ID token - v1.0 format. Graph issues v1.0 tokens regardless of what your app requested.

Same lifetime here; Entra may vary it.

Authentication context class - 1 means password-level.

Which client obtained this token. Your `client_id` appears here, not in `aud`.

How the client authenticated to Entra: 0 = public client, no secret. 1 = client secret. 2 = certificate.

Delegated scopes granted. Present because a user signed in. An app-only token would have roles instead and no `scp`.

Same `oid` as the ID token. This is how you know it is the same human.

A DIFFERENT `sub` from the ID token. `sub` is pairwise per audience, and the audience here is Graph, not your app.

User principal name.

KEY POINT - Read the Two `aud` Lines Again

Same person, same app, same second - and two tokens with different issuers, different audiences, and different subjects. The ID token's `aud` is your client. The access token's `aud` is Graph. Your client is named in the access token, but only as `appid` - the party that asked for it, not the party it is for. That is the whole lesson in two lines of JSON. The nonce appears only in the ID token because only the ID token is meant for the party that generated the nonce.

The Confused Deputy Problem (CWE-441)

A confused deputy is a component with privileges that is tricked into using them on behalf of someone who should not have them. In the token context: if an API accepts an access token whose aud names a different application, it is accepting a credential that was not issued for it. An attacker who obtains a valid access token for application A can present it to application B if B does not validate aud. B - the confused deputy - performs operations it would never have authorized had the request come directly from the attacker. Microsoft's documentation names this explicitly and states that APIs must only accept tokens whose aud matches one of the API's own AppId URIs.

Who Validates What - A Common Exam Question

- **The client validates the ID token:** verifies the signature against the issuer's published public keys, then checks iss, aud (must equal its client_id), exp, iat, and nonce.
- **The resource server validates the access token:** verifies the signature, then checks iss, aud (must equal the API's AppId URI), exp, and the scopes or roles relevant to the operation.
- **The client does not validate the access token:** it treats it as opaque and forwards it to the API.

An application that accepts access tokens meant for another resource is wrong. An application that uses ID tokens to call APIs is wrong. Both will appear to work - until they do not.

STOP - Answer these before reading on. Out loud or on paper. No looking back.

1. Cover the annotations. Point to the line in each token that tells you who it is for, and the line that tells you who asked for it. Are they the same line?
2. Your API receives the Graph access token above by mistake. Walk through the validation steps and name the exact check that rejects it.
3. Two tokens, two different sub values, one human. Which claim proves it is one human, and why does Entra deliberately make sub differ?

VERIFY - Do not trust this document. Check it.

OpenID Connect Core 1.0 §2 (ID Token - the claim list) and §3.1.3.7 (ID Token Validation - the ordered checklist the client must run). Microsoft Learn: 'ID token claims reference' and 'Access token claims reference' - confirm the appidacr values and the pairwise sub behaviour. MITRE CWE-441 for the confused deputy definition.

SECTION 07

Synthesis - The Two Eras and Why Conditional Access Exists

We have covered five protocols spanning four decades. Put the two eras side by side and draw the conclusion the entire rest of Phase 1 depends on.

	Kerberos	OAuth 2.0 / OIDC
Trust anchor	A shared secret with the KDC	A signature over a JWT, verified using public keys fetched from the issuer's JWKS endpoint
Token audience	Cryptographically bound to one service - encrypted with that service's key	The aud claim: a string, checked by the recipient
Possession model	Ticket is useless without the session key - the authenticator proves possession	Bearer. Possession is sufficient.
Network assumption	Hostile network, trusted endpoints, single realm	Hostile everything - network, endpoint, and client are all untrusted
Failure mode	Offline cracking of the service account key (Kerberoasting); pass-the-ticket	Token theft and replay

The Trade-Off That Changed Everything

Kerberos was built for a network you did not trust, with endpoints and a directory you did. One domain. All your servers in it. The KDC on-premises and physically secured.

Cloud identity was built for a world where the network, the endpoint, and the client application are all suspect - anyone, anywhere, on any device, accessing applications run by multiple organizations. To reach that world, the protocols paid a price: bearer tokens.

A bearer access token, once issued, is valid until it expires. Outside of the narrow exception covered below, the issuer has no way to recall it. If that token is stolen - pulled from browser storage, intercepted in transit, copied from a log - the thief can use it until expiry, from any location, on any device, with no further authentication.

WHY CONDITIONAL ACCESS EXISTS

Conditional Access is a pre-issuance gate. It decides whether to issue a token at all, and what restrictions to place on the session. A stolen bearer token is a post-issuance fact - Conditional Access has no power over it. This is the seam that Topic 3 lives in. Write this down: Conditional Access is pre-issuance. Token theft is post-issuance. They exist at different moments in time, and a policy that looks correct in the portal may do nothing to stop token replay.

Continuous Access Evaluation - Narrowing the Gap

CAE is Microsoft's partial answer to the post-issuance problem, and the one exception to 'the issuer cannot recall a token.' For a small set of critical events - account disabled, password changed, explicit token revocation, elevated user risk - Entra ID notifies CAE-capable resource servers in near-real-time, and they reject the token before it expires.

CAE does not eliminate the bearer token problem. It shrinks the window for a specific set of high-priority events on resources that support it. For everything else, the token is valid until expiry.

STOP - Answer these before reading on. Out loud or on paper. No looking back.

1. Using only Figures 1 and 4: at which arrow does Conditional Access act, and at which arrow does a stolen bearer token get used? Why can CA not see the second one?
2. Kerberos has an authenticator. What is the OAuth-world equivalent, and why is it not the default?

VERIFY - Do not trust this document. Check it.

Microsoft Learn: 'Continuous access evaluation' - confirm the list of critical events and which resources support it. The claim that bearer tokens are *the* reason CA and CAE exist is this document's reasoning about design intent, not a Microsoft statement. Treat it as a strong argument, not a finding.

Review - What You Must Be Able to Explain Without Notes

The concept quiz tests whether you can explain mechanisms, not whether you can recite definitions. Before the quiz, you must be able to explain each of the following in plain language to someone who has never seen these protocols.

Kerberos

- Why a service can verify a Kerberos ticket without contacting the KDC
- What the TGT contains and who can decrypt it
- What the service ticket contains and why only the target service can decrypt it
- What the authenticator is and why it makes a Kerberos ticket proof-of-possession rather than bearer
- What an SPN is and what happens when one is missing or duplicated
- Why Kerberoasting works and which encryption type makes it cheap
- What msDS-SupportedEncryptionTypes controls and which value means AES-only

NTLM

- What the three NTLM messages are and what each contains
- Why NTLM has no mutual authentication and what that enables
- Why NTLM has no target binding, what relay is, and what EPA does about it
- What pass-the-hash means and why the hash is sufficient
- Why NTLM is still running in most environments even when Kerberos is configured

SAML 2.0

- What problem SAML solves that Kerberos cannot
- What the three parties are and why the browser is the intermediary
- What the trust anchor is and why certificate rotation breaks SAML
- What an assertion contains and what the audience restriction is for
- What XML signature wrapping is and how it works

OAuth 2.0

- The single most important thing OAuth 2.0 is not
- The four roles and what each does
- What the authorization code flow does and why PKCE is required
- What a bearer token is and the security implication of that model
- Why a client should not validate or parse an access token

OpenID Connect

- What OIDC adds to OAuth 2.0 and what the ID token is for
- The five required ID token claims, the conditional sixth, and what each is for
- Who validates the ID token and who validates the access token - and that both start with the signature
- What the confused deputy problem is in the token context
- Given two decoded tokens, point to aud, appid, sub, oid and explain each

Synthesis

- How bearer tokens differ from Kerberos service tickets in what possession grants
- Why Conditional Access is a pre-issuance gate and what that means for token theft
- What CAE is and what it does - and does not - solve

CRITICAL - Cold Recall Opens Every Session

The first five minutes of every attended session is cold recall - no notes, no document. You will be asked to explain one or two mechanisms from a previous topic without preparation. If you cannot explain it without notes, the XP for that topic is at risk of deduction. Understanding, not memorization. Mechanisms, not definitions.

SECTION 09

Lab - Build, Then Explain the Delta

Your five predictions were sealed in Section 01. Now build. Use the employer dev tenant only - confirmed prod-isolated, no shared identities. This lab contains no offensive technique: app registration and token inspection only. No manager conversation is required (Topic 3 is where that applies).

Part B - Build

- Register an application in the dev tenant. Note its client_id.
- Run the authorization code + PKCE flow with scopes openid profile User.Read. Capture both tokens.
- Decode both at jwt.ms or by base64url-decoding the middle segment. Lay them next to the two tokens in Section 06.
- Record for each token: aud, iss, scp or roles, nonce, exp, appid, sub, oid.
- Repeat with only openid profile. Record what changes.
- If you can, obtain an app-only token with client credentials for the same app registration and compare the scp/roles claims.

Part C - Explain the Delta

For every prediction that was wrong, you must explain:

- What model produced the wrong prediction
- What the correct model is
- Why the two models differ

'I guessed' is not an explanation. A wrong prediction you understand is worth more than a right prediction you cannot explain. Bring the sealed prediction file and Part C to the session.

Forensic Lab - 50 XP

Separate from the build lab. Pull the sign-in log entries for the sign-ins you just performed from the Entra portal. For each entry, identify: which application, which resource, interactive or non-interactive, which authentication method satisfied the request, and whether a Conditional Access policy was evaluated. Then answer in writing: which of these log fields correspond to which claims in the tokens you decoded? Which token claim has no corresponding log field, and why?

SECTION 10

Artifact - Authentication Path Inventory and Legacy Protocol Exposure Map

Worth 40 XP. Must contain real organizational context - a hypothetical earns zero. It is also a candidate founding document, the kind a 2027 infrastructure team would inherit and use.

What the Document Contains

One row per authentication path in your estate. Suggested columns:

- System - the name of the system or service
- Protocol in use - Kerberos / NTLM / SAML / OIDC / mixed
- If Kerberos: the etype actually observed in event logs
- Any msDS-SupportedEncryptionTypes override - who set it and why
- If SAML: the signing certificate's expiry date
- Evidence I can produce today - the log, event, or attribute that backs the claim

Read-Only Data Sources - No Configuration Change Required

- Events 4768 (TGT requested) and 4769 (service ticket requested) on the domain controllers - observed Kerberos etypes
- Get-ADObject -Filter * -Properties msDS-SupportedEncryptionTypes - per-account overrides
- Microsoft's Kerberos-Crypto tooling on GitHub: List-AccountKeys.ps1 and Get-KerbEncryptionUsage.ps1 -Encryption RC4
- Entra sign-in logs - which applications are authenticating via OIDC versus SAML

Why This Is Time-Sensitive

Microsoft's RC4 enforcement deadline is behind you - July 2026. Any account still carrying an explicit RC4 override is an exception someone set to stop a breakage in April 2026 when enforcement first activated. If nobody documented why, that exception is folklore. A 2027 infrastructure team either inherits folklore, or inherits a table with an owner and a reason attached. You are writing the table.

KEY POINT - Scope It Small - Done Properly Beats Half the Estate

Start with one system class - the domain controllers, or the Entra-federated apps, or the SQL Server service accounts. Do that one class completely and correctly. A single system class done properly with real evidence is worth 40 XP. The entire estate half-done with no evidence trail is worth zero.

SECTION 11

Quick Reference - One Page

The condensed version of everything in this document. Use it for review before the quiz, not as a substitute for the sections.

Protocol	What It Solves	Trust Anchor	Token Model	Key Failure Mode
Kerberos	AuthN inside a domain without sending the password	Shared key with the KDC	Service ticket + authenticator - proof-of-possession	Kerberoasting; pass-the-ticket
NTLM	AuthN when Kerberos is unavailable	Password hash	Challenge-response - deprecated	Relay; pass-the-hash
SAML 2.0	AuthN across organizations with no shared directory	IdP public certificate, exchanged out of band	Signed XML assertion	Certificate rotation; XML signature wrapping
OAuth 2.0	Authorization: may this app act on this resource for this user?	AS public keys (JWKS)	Bearer access token	Token theft and replay
OIDC	Authentication layer on top of OAuth 2.0	AS public keys (JWKS)	Bearer ID token + access token	Confused deputy - wrong token for the wrong audience

The Five Things You Must Have Memorized

- **1.** OAuth 2.0 is an authorization framework, not an authentication protocol. OIDC is the authentication layer.
- **2.** The ID token is for the client and says who the user is. The access token is for the API and says what may be done. They are not interchangeable.
- **3.** A bearer token: possession is sufficient. A Kerberos service ticket: possession is not sufficient - the presenter must also prove it holds the session key via the authenticator.
- **4.** Conditional Access is pre-issuance. Token theft is post-issuance. CA cannot stop replay of a token it has already issued; only CAE partially can, for specific events.
- **5.** Kerberoasting: any authenticated domain user can request a service ticket for any SPN, take the encrypted blob offline, and crack the service account password. No alert fires.

Sources: RFC 4120 (Kerberos), RFC 6749 (OAuth 2.0), RFC 7636 (PKCE), RFC 9700 (BCP 240, January 2025), OpenID Connect Core 1.0, OASIS SAML 2.0 Core and Profiles, MS-NLMP, Microsoft Learn SC-300 study guide (verified April 27,

2026), *Microsoft Learn deprecated features (NTLM)*, *Microsoft Support CVE-2026-20833 (RC4 removal)*, *MITRE CWE-441*.